

CSE 302

Lecture 3, Software Engineering

- ▶ Dr. Ashraf Mahroos, Ph.D
- ▶ Professor of Engineering Systems
- ▶ The figures and text included in slides are borrowed from various books, websites, and other sources for academic purposes only. The author do not claim any originality.

Topics covered



- ✧ Software process models
- ✧ Process activities
- ✧ Coping with change
- ✧ Process improvement

The software process



- ✧ *Software process*: a structured set of activities required to develop a software system
- ✧ Many different software processes but all involve:
 - *Specification* – defining what the system should do;
 - *Design and implementation* – defining the organization of the system and implementing the system;
 - *Validation* – checking that it does what the customer wants;
 - *Evolution* – changing the system in response to changing customer needs.
- ✧ A *software process model* is an abstract representation of a process. It presents a description of a process from some particular perspective.

Plan-driven and agile processes



- ✧ **Plan-driven processes** are processes where all of the process activities are planned in advance and progress is measured against this plan.
- ✧ In **agile processes**, planning is incremental and it is easier to change the process to reflect changing customer requirements.
- ✧ In practice, most practical processes include elements of both plan-driven and agile approaches.
- ✧ There are no right or wrong software processes.

Software process models



✧ The waterfall model (1)

- Plan-driven model. Separate and distinct phases of specification and development.

✧ Incremental development (2)

- Specification, development and validation are interleaved. May be plan-driven or agile.

✧ Integration and configuration (3)

- The system is assembled from existing configurable components. May be plan-driven or agile.

- ✧ In practice, most large systems are developed using a process that incorporates elements from all of these models.

Generic Software Process Models

1 The waterfall model

- Separate and distinct phases of specification and development

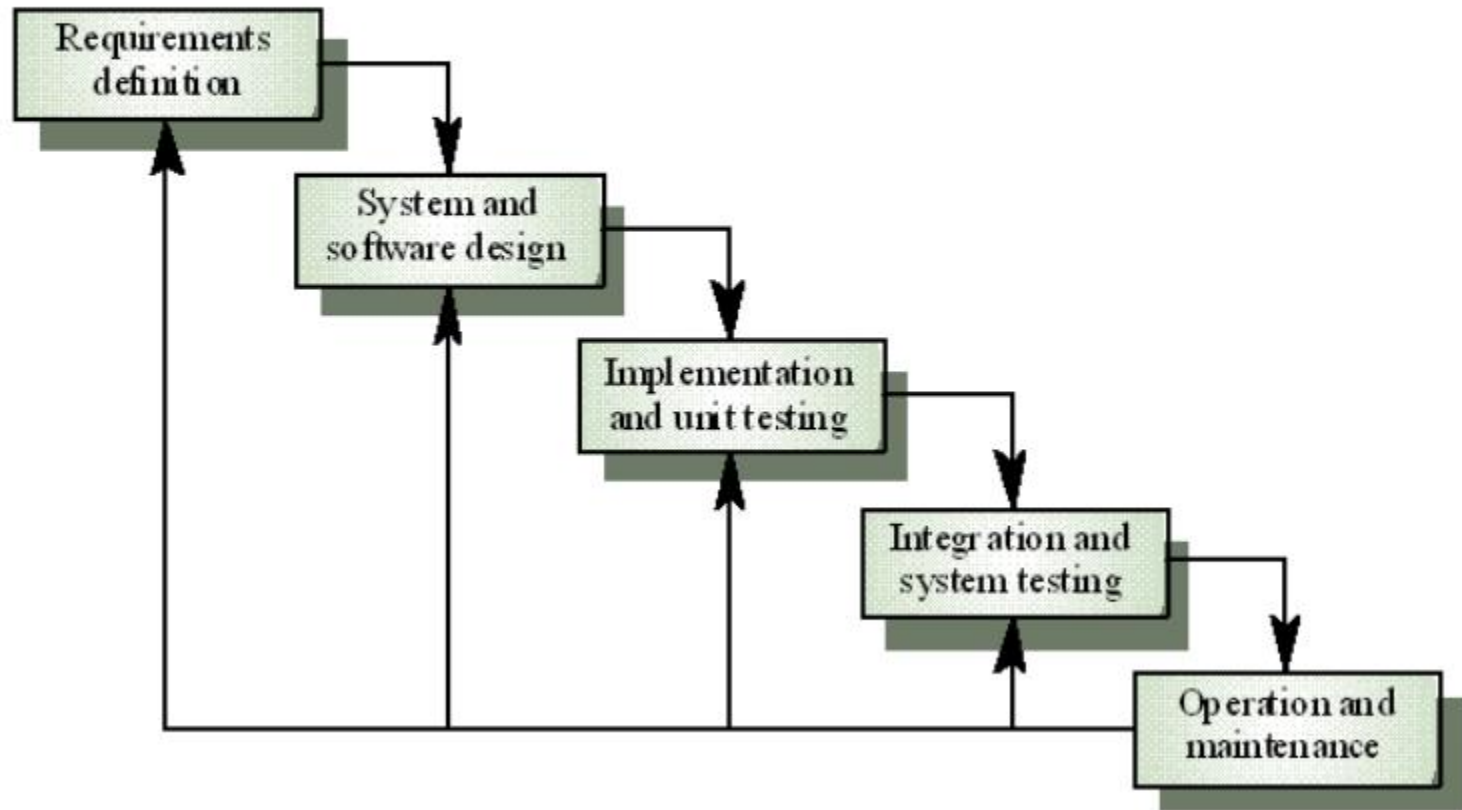
1 Evolutionary development

- Specification and development are interleaved

1 Reuse-based development

- The system is assembled from some (most likely) or all existing components

Waterfall Model



Advantages and Disadvantages of Waterfall Methodology

Advantages

For Users	For Employees
Longer operational life	Less need for supporting systems
Meets user needs and requirements	Excellent documentation
More flexible	

Disadvantages

For Users	For Employees
More expensive to maintain	Requires extensive research into user needs
More difficult to use	Difficult to customize
	Requires more training for employees

Evolutionary Development

Two general types:

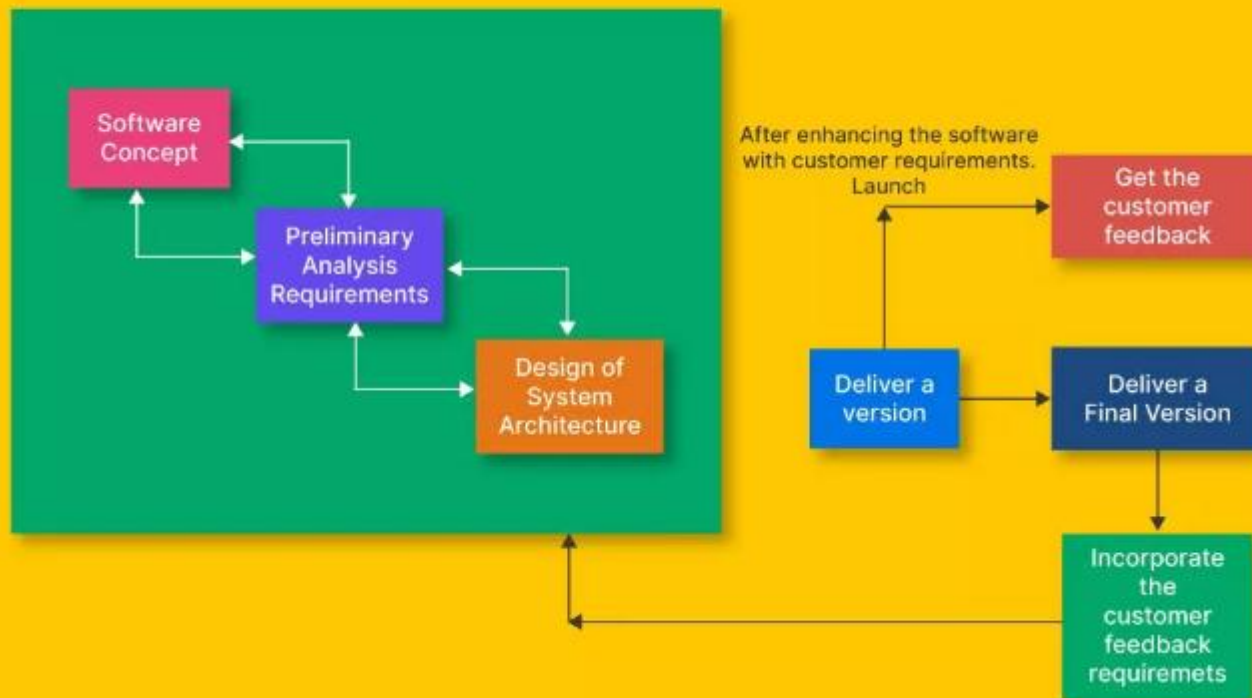
1 **Exploratory development**

- Objective is to work with the customers to evolve a final system from an initial outline specification. Process starts with the well-understood requirements.

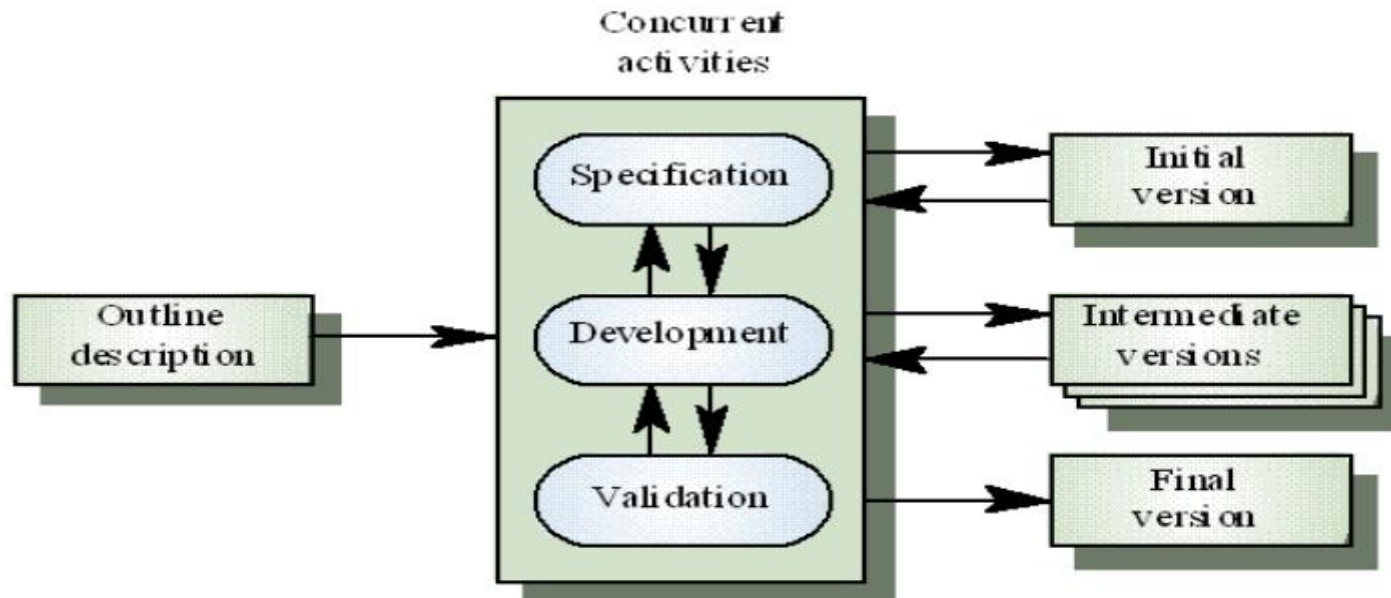
1 **Prototyping (throw-away?)**

- Objective is to understand the system requirements. Process starts with the poorly understood requirements.

Evolutionary Model in Software Engineering



Evolutionary Development



Here are the other major advantages and disadvantages of the evolutionary model in software engineering:

Advantages	Disadvantages
Allows for incremental development and improvement	May require more time and effort to manage and coordinate changes
Enables early delivery of working software	Requires a high level of collaboration and communication among team members
Facilitates continuous feedback and collaboration with stakeholders	May not be suitable for projects with strict deadlines or fixed budgets
Reduces the risk of project failure by identifying issues early on	Can result in a less predictable and more uncertain development process

Types of Evolutionary Models in Software Engineering

The three main types of evolutionary models are:

- Prototype model
- Spiral model
- Concurrent model

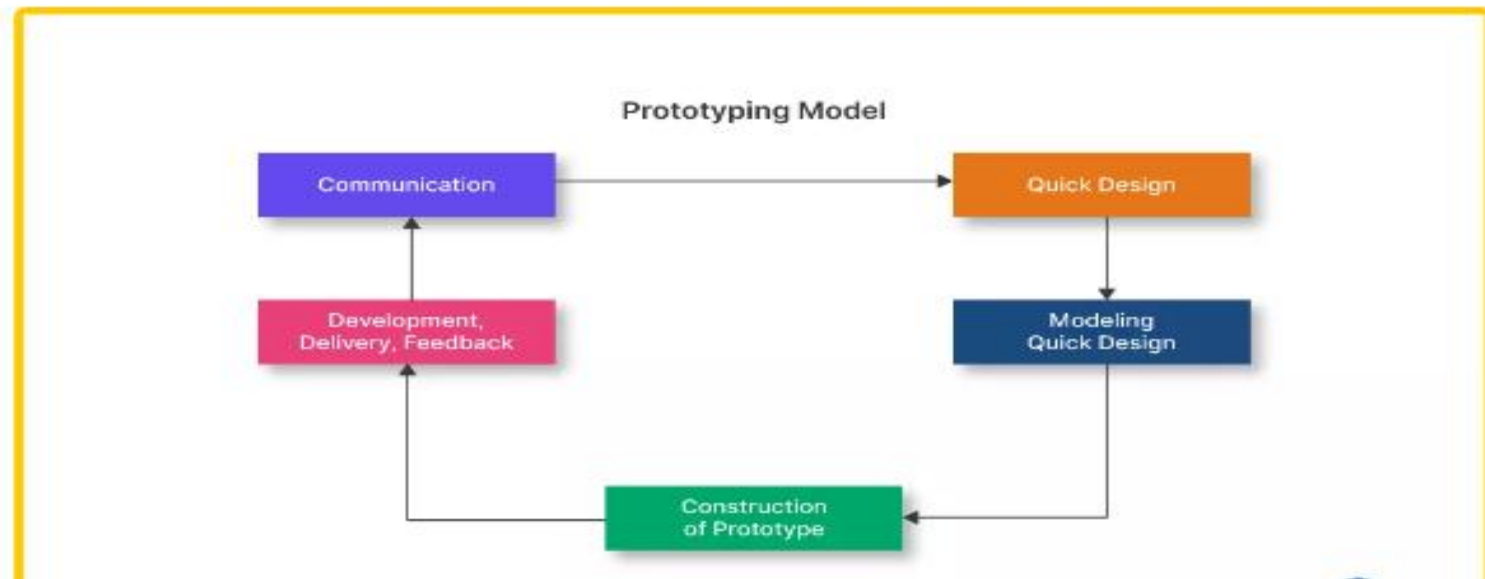
Let's study these models, and their advantages and disadvantages in detail

1. Prototyping Model

The prototype model is a software development approach that involves creating an initial, working version of a product to gather feedback and make improvements. This model is particularly useful when requirements are not well-defined or may change throughout the development process. The prototype serves as a visual representation of the final product, allowing stakeholders to provide feedback and suggest changes before the full development begins. It helps identify potential issues early on and reduces the risk of developing a product that does not meet the user's needs. The prototype model promotes collaboration and iterative development, with multiple iterations of prototyping and refining until the final product is achieved.

Here are the major phases of a prototyping model:

- **Communication:** Developer and customer interact on the objectives for software products.
- **Quick Design:** A detailed plan is not visible; rather this phase helps the developer to work on the prototype.
- **Modeling Quick Design:** Software is developed, allowing the developer to understand the exact niche requirements.
- **Construction of prototype:** Customer evaluates the prototype
- **Delivery time, deployment, and feedback:** The application is launched and user feedback is received. Post this, issues, if any, are fixed by the developer team.



Advantages of Prototyping Model

1. In this model, the stakeholder/customer role plays a vital role in the development of the application. They contribute to it through, -->Customer response, Customer evaluation, and customer feedback where they intimate issues to the development team. The developer team can then sort out the problem.
2. Resolving response time is quick.
3. Project risk can be avoided and the project team can improvise according to the feedback from users.
4. This model allows a greater degree of customer involvement, which helps improve product quality.

Disadvantages of the Prototyping Model

1. Time-consuming: Multiple tickets reported by the customer can be tedious for the development team to cope with a rhythm, leading to slow evolvement sometimes.
2. Scope: If the prototype is constantly changing, it can be difficult to manage the scope of the project.
3. There is a risk that the prototype will become the final product, even if it is not satisfactory to the users and stakeholders.

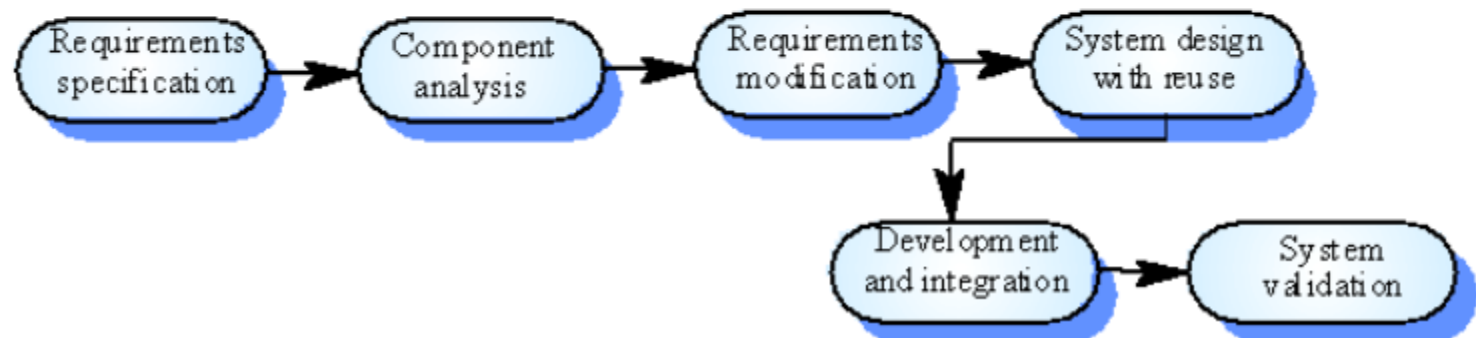
Example of a prototype model:

An example of a prototype model is a mock-up of a new mobile application. Let's say a software development team is tasked with creating a new social media app. Before investing time and resources into building the full application, they may create a prototype model to test the concept and gather feedback. This prototype model could be a simplified version of the app with basic features and a user interface that allows users to navigate through different screens. The team can then share this prototype with potential users or stakeholders to get their input and make any necessary adjustments before progressing with the development process. By using a prototype model, the team can save time and resources by identifying any potential issues or improvements early on in the design phase.

Reuse-oriented Development

- 1 Based on systematic reuse where systems are integrated from existing components or COTS (commercial-off-the-shelf) systems
- 1 This approach is becoming more important, but there is still limited experience with it.

Reuse-oriented Development



There are three types of software component that may be used in a reuse-oriented process:

1. Web services that are developed according to service standards and which are available for remote invocation.
2. Collections of objects that are developed as a package to be integrated with a component framework such as .NET or J2EE.
3. Stand-alone software systems that are configured for use in a particular environment.

Advantages :

- It can reduce total cost of software development.
- The risk factor is very low.
- It can save lots of time and effort.
- It is very efficient in nature.

Disadvantages :

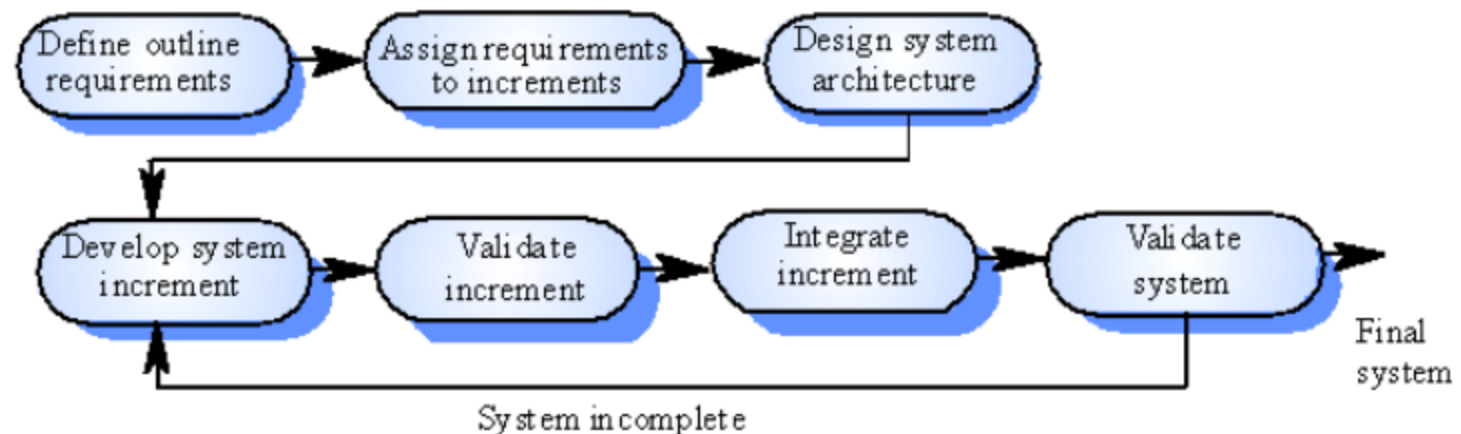
- Reuse-oriented model is not always worked as a practice in its true form.
- Compromises in requirements may lead to a system that does not fulfill requirement of user.
- Sometimes using old system component, that is not compatible with new version of component, this may lead to an impact on system evolution

Process Iteration

- 1 System requirements ALWAYS evolve in the course of a project. So, process iteration where earlier stages are reworked is always part of the process, especially for large systems.
- 1 Iteration can be applied to any of the generic process models.
- 1 Examples of two iterative approaches:
 - Incremental development
 - Spiral development

Incremental Development

- 1 Rather than deliver the system as a single delivery, the development and delivery is broken down into increments with each increment delivering part of the required functionality.
- 1 User requirements are prioritized and the highest priority requirements are included in early increments.
- 1 Once the development of an increment is started, the requirements are frozen, though requirements for later increments can continue to evolve.



Incremental Development Advantages

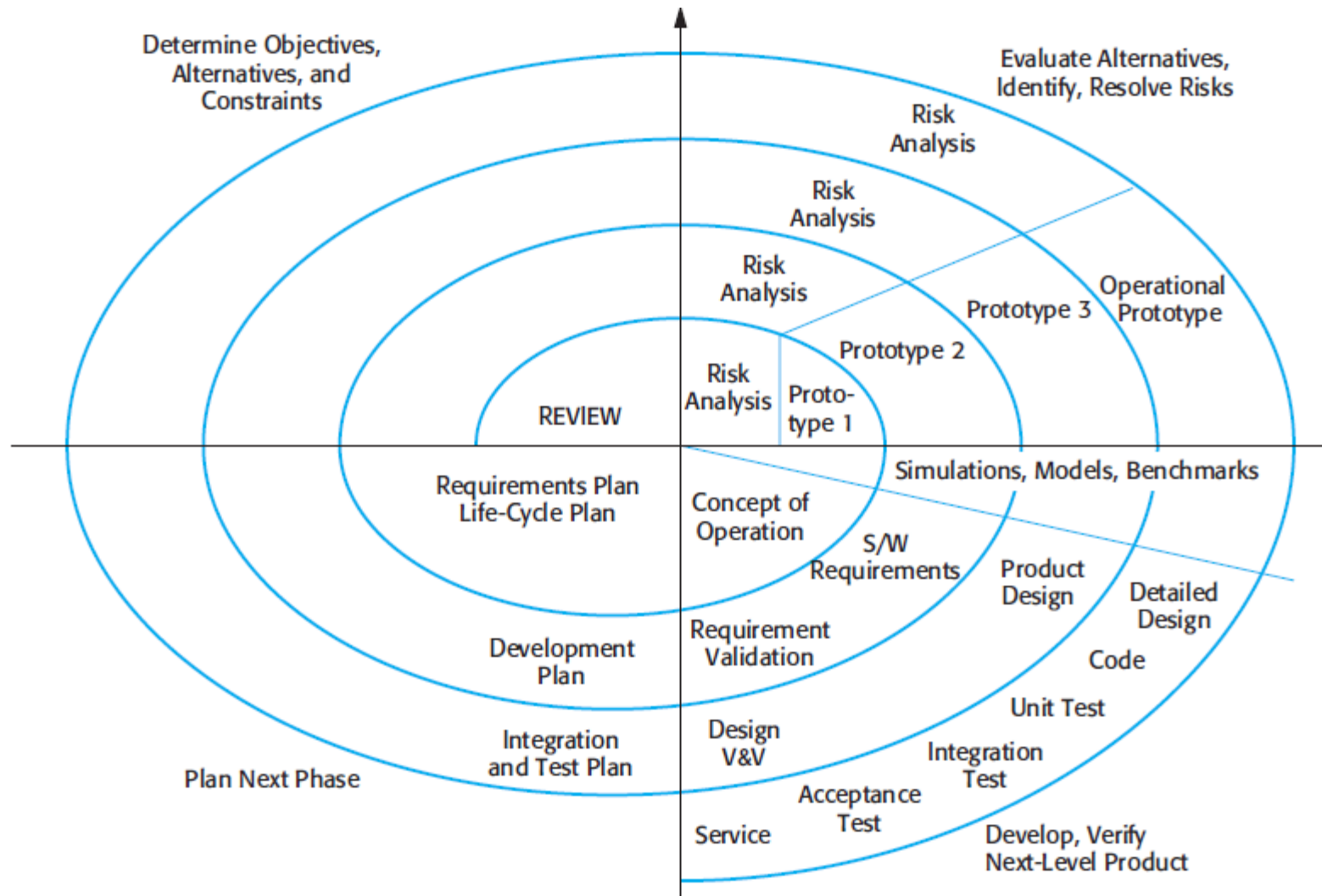
- 1 Customers do not have to wait until the entire system is delivered until they can gain value from it.
- 1 Early increments act as a prototype to help elicit requirements for later increments.
- 1 Lower risk of overall project failure
- 1 The highest priority system services tend to receive the most testing.

Spiral Development

- 1 Process is represented as a spiral rather than as a sequence of activities with backtracking
- 1 Each loop in the spiral represents a phase in the process.
- 1 No fixed phases such as specification or design - loops in the spiral are chosen depending on what is required
- 1 Risks are explicitly assessed and resolved throughout the process.

Boehm's spiral model

A risk-driven software process framework (the spiral model) was proposed by Boehm (1988). This is shown in Figure 2.11. Here, the software process is represented as a spiral, rather than a sequence of activities with some backtracking from one activity to another. Each loop in the spiral represents a phase of the software process. Thus, the innermost loop might be concerned with system feasibility, the next loop with requirements definition, the next loop with system design, and so on.



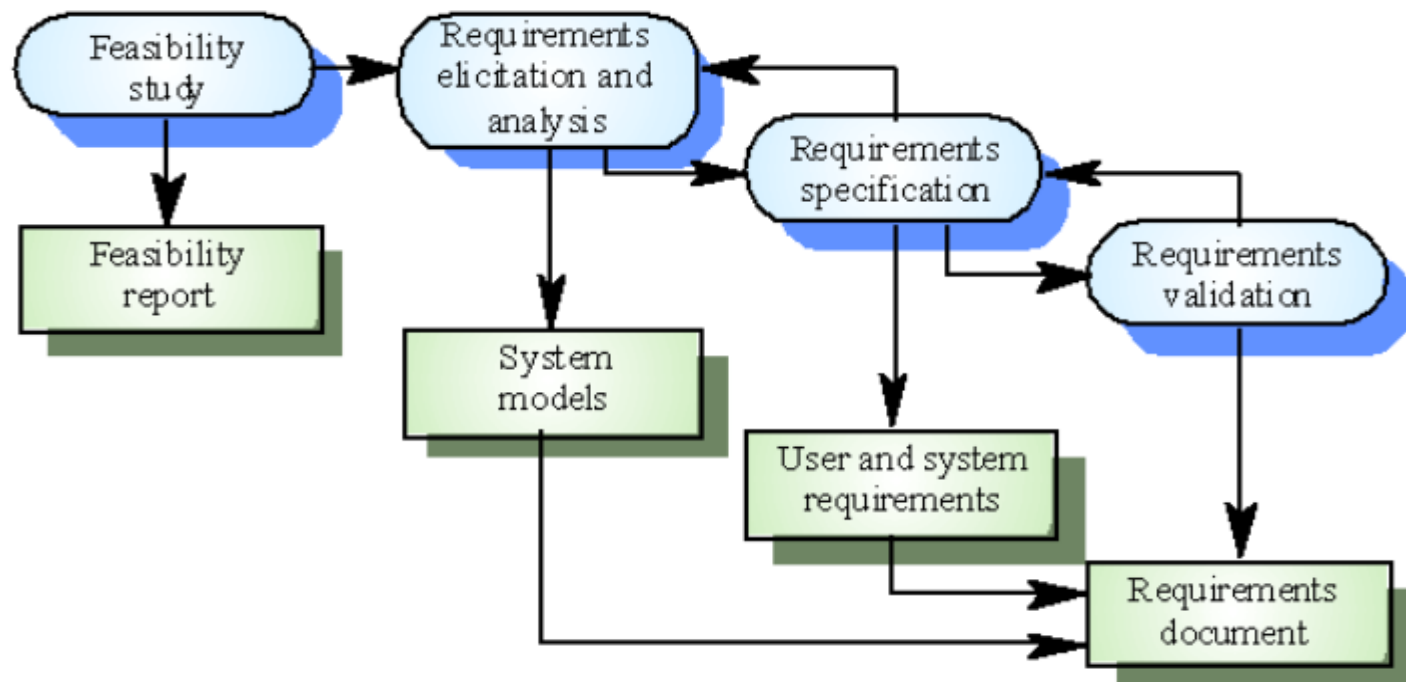
1. *Objective setting* Specific objectives for that phase of the project are defined. Constraints on the process and the product are identified and a detailed management plan is drawn up. Project risks are identified. Alternative strategies, depending on these risks, may be planned.
2. *Risk assessment and reduction* For each of the identified project risks, a detailed analysis is carried out. Steps are taken to reduce the risk. For example, if there is a risk that the requirements are inappropriate, a prototype system may be developed.
3. *Development and validation* After risk evaluation, a development model for the system is chosen. For example, throwaway prototyping may be the best development approach if user interface risks are dominant. If safety risks are the main consideration, development based on formal transformations may be the most appropriate process, and so on. If the main identified risk is sub-system integration, the waterfall model may be the best development model to use.
4. *Planning* The project is reviewed and a decision made whether to continue with a further loop of the spiral. If it is decided to continue, plans are drawn up for the next phase of the project.

Requirements Engineering

The process of establishing

- 1 what services are required of the system
- 1 the constraints on the system's operation and development

The Requirements Engineering Process



There are four main activities in the requirements engineering process:

1. *Feasibility study* An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies. The study considers whether the proposed system will be cost-effective from a business point of view and if it can be developed within existing budgetary constraints. A feasibility study should be relatively cheap and quick. The result should inform the decision of whether or not to go ahead with a more detailed analysis.
2. *Requirements elicitation and analysis* This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, task analysis, and so on. This may involve the development of one or more system models and prototypes. These help you understand the system to be specified.
3. *Requirements specification* Requirements specification is the activity of translating the information gathered during the analysis activity into a document that defines a set of requirements. Two types of requirements may be included in this document. User requirements are abstract statements of the system requirements for the customer and end-user of the system; system requirements are a more detailed description of the functionality to be provided.
4. *Requirements validation* This activity checks the requirements for realism, consistency, and completeness. During this process, errors in the requirements document are inevitably discovered. It must then be modified to correct these problems.

Software Design and Implementation

The process of converting the system specification into an executable system.

1 **Software design**

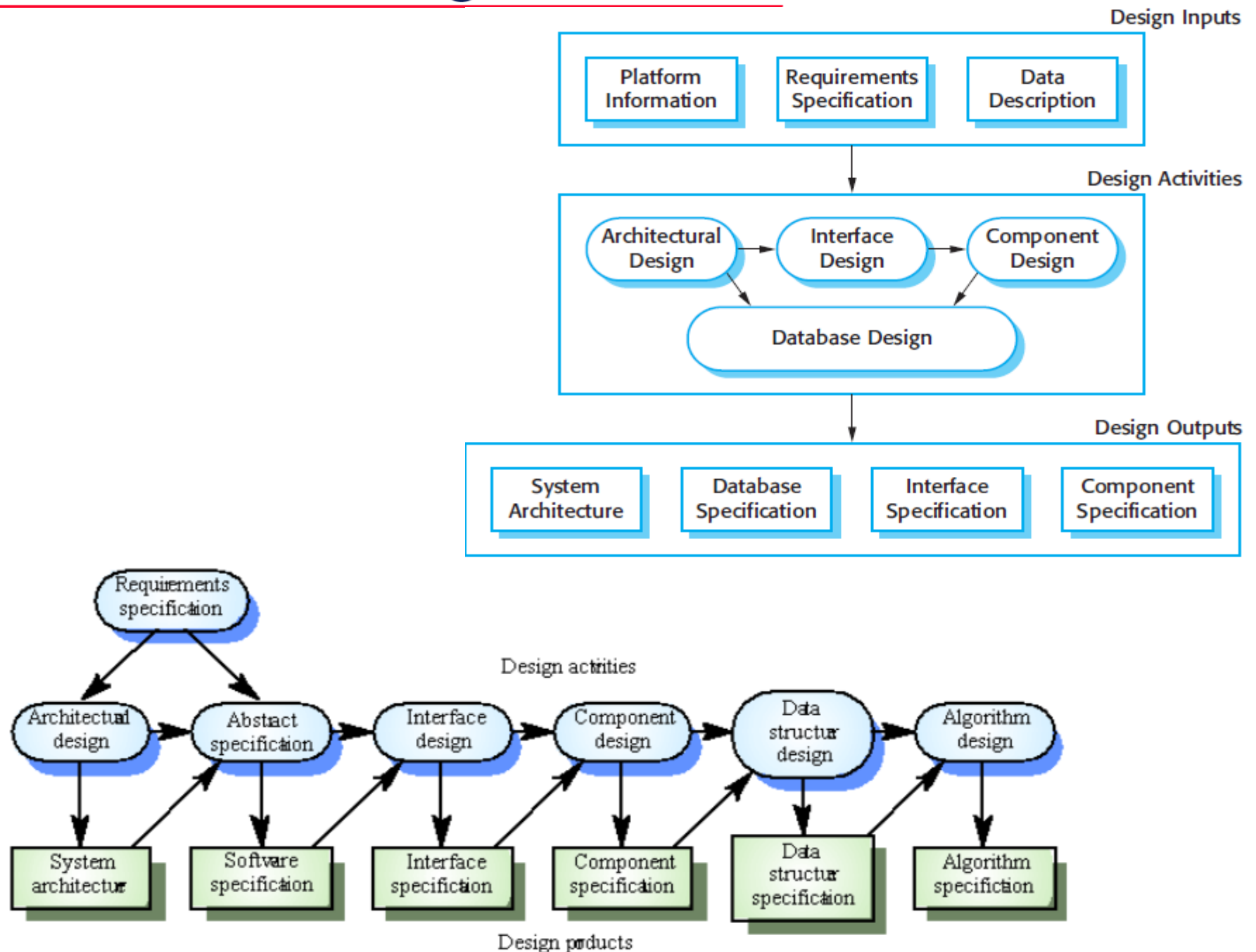
- Design a software structure that realizes the specification

1 **Implementation**

- Translate this structure into an executable program

1 **The activities of design and implementation are closely related and may be inter-leaved.**

The Software Design Process



Design Methods

Systematic approaches to developing a software design

- 1 The design is usually documented as a set of graphical models.
- 1 Possible models
 - Data-flow model
 - Entity-relation-attribute model
 - Structural model
 - Object models

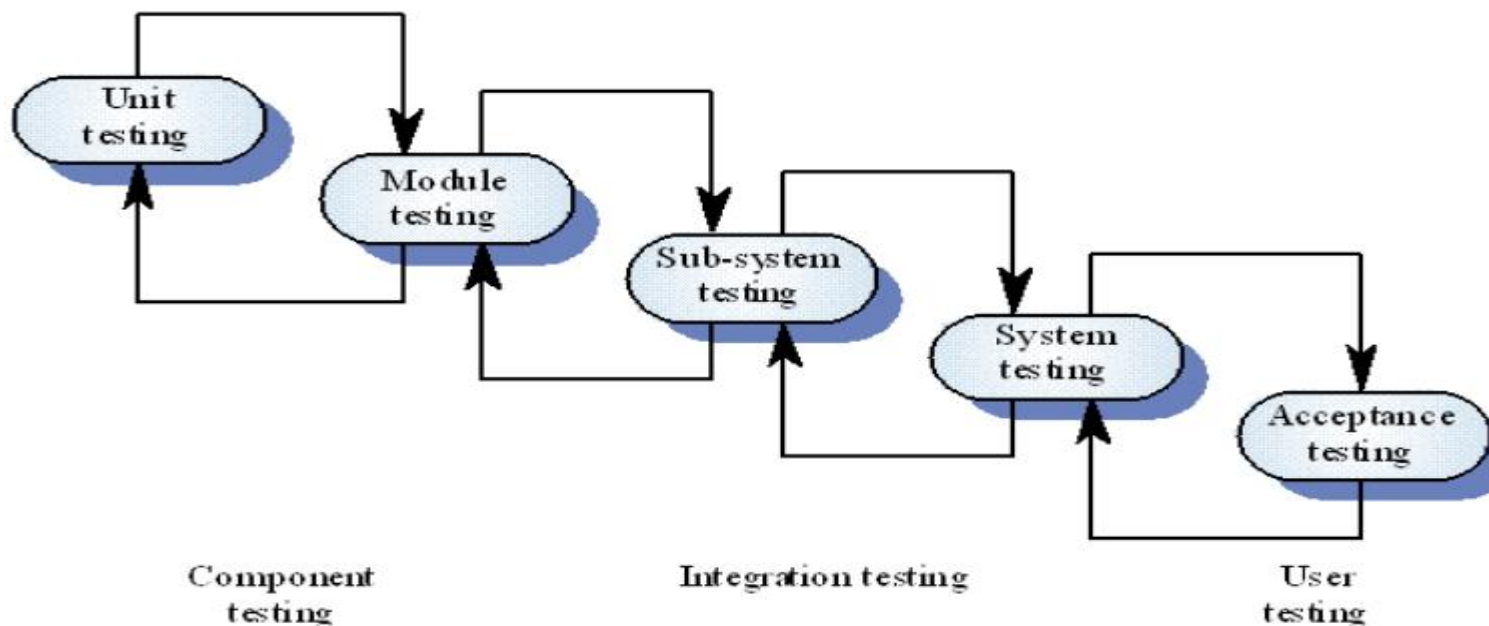
Programming and Debugging

- 1 Translating a design into a program and removing errors from that program
- 1 Programming is a personal activity - there is no generic programming process.
- 1 Programmers carry out some program testing to discover faults in the program and remove these faults in the debugging process.

Software Validation

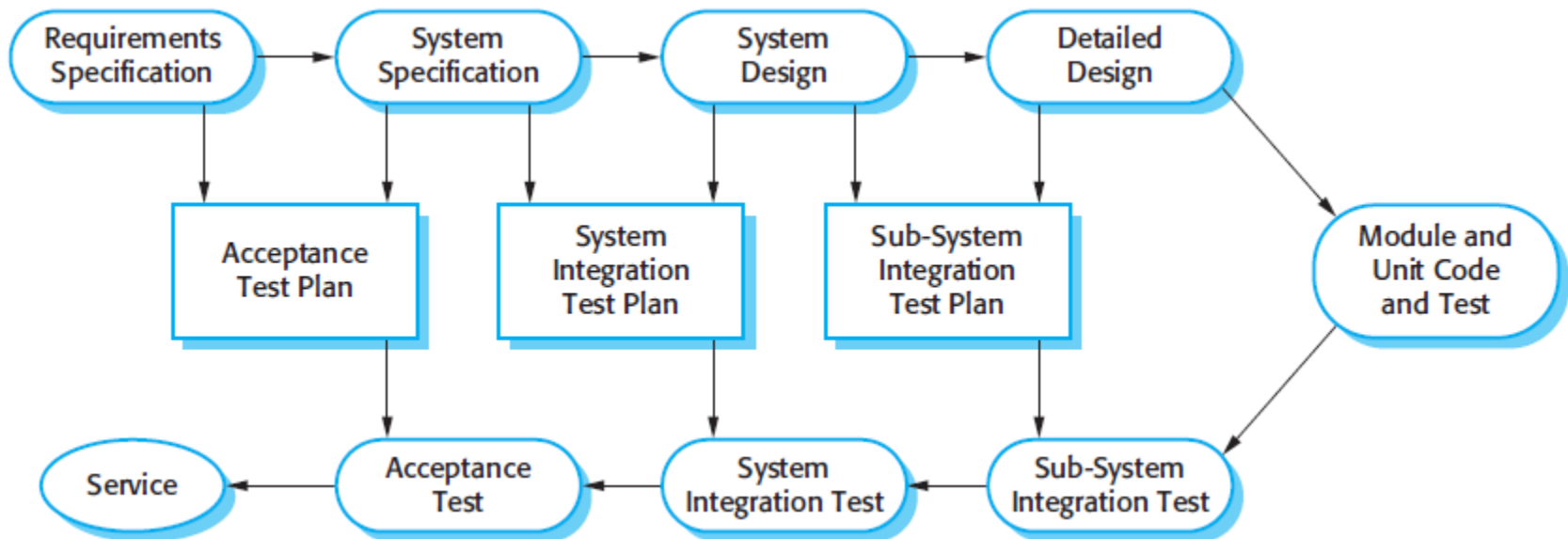
- 1 Verification and validation is intended to show that a system conforms to its specification and meets the requirements of the system customer.
- 1 Involves checking and review processes and system testing
- 1 System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.

The Testing Process



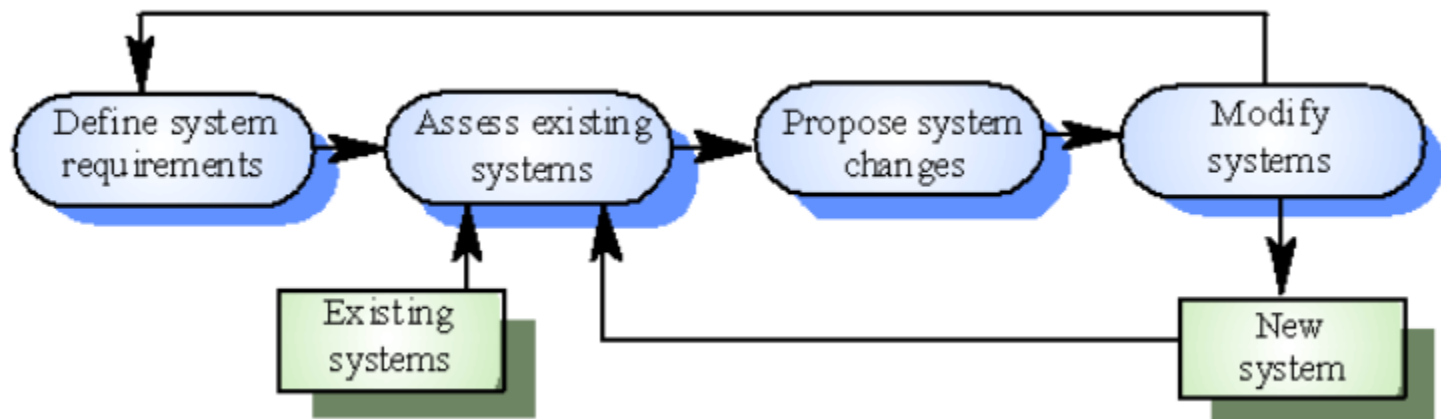
Testing Stages

- 1 **Unit testing**
 - Individual components are tested
- 1 **Module testing**
 - Related collections of dependent components are tested
- 1 **Sub-system testing**
 - Modules are integrated into sub-systems and tested. The focus here should be on interface testing
- 1 **System testing**
 - Testing of the system as a whole. Testing of emergent properties
- 1 **Acceptance testing**
 - Testing with customer data to check that it is acceptable



Software Evolution

- 1 Software is inherently flexible and can change (as opposed to hardware).
- 1 In the past, there has been a demarcation between development and evolution (maintenance). This is increasingly irrelevant as fewer and fewer systems are completely new.
- 1 Software engineering should be thought of as an evolutionary process where software is continually changed over its lifetime in response to customer needs.



This distinction between development and maintenance is increasingly irrelevant. Hardly any software systems are completely new systems and it makes much more sense to see development and maintenance as a continuum. Rather than two separate processes, it is more realistic to think of software engineering as an evolutionary process (Figure 2.8) where software is continually changed over its lifetime in response to changing requirements and customer needs.

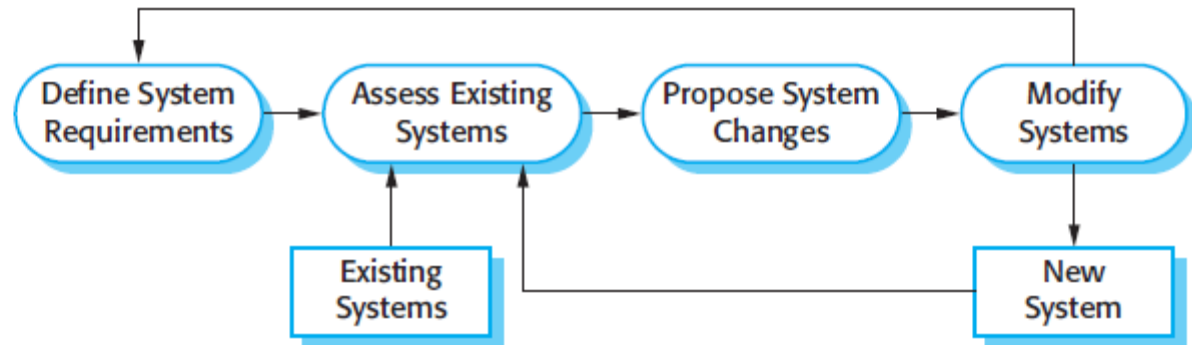


Figure 2.8 System evolution

2.3 Coping with change

Change is inevitable in all large software projects. The system requirements change as the business procuring the system responds to external pressures and management priorities change. As new technologies become available, new design and implementation possibilities emerge. Therefore whatever software process model is used, it is essential that it can accommodate changes to the software being developed.

Two ways of coping with change and changing system requirements.

1. **System prototyping**, where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of some design decisions. This supports change avoidance as it allows users to experiment with the system before delivery and so refine their requirements. The number of requirements change proposals made after delivery is therefore likely to be reduced.
2. **Incremental delivery**, where system increments are delivered to the customer for comment and experimentation. This supports both change avoidance and change tolerance. It avoids the premature commitment to requirements for the whole system and allows changes to be incorporated into later increments at relatively low cost.

Figure 2.9 The process of prototype development

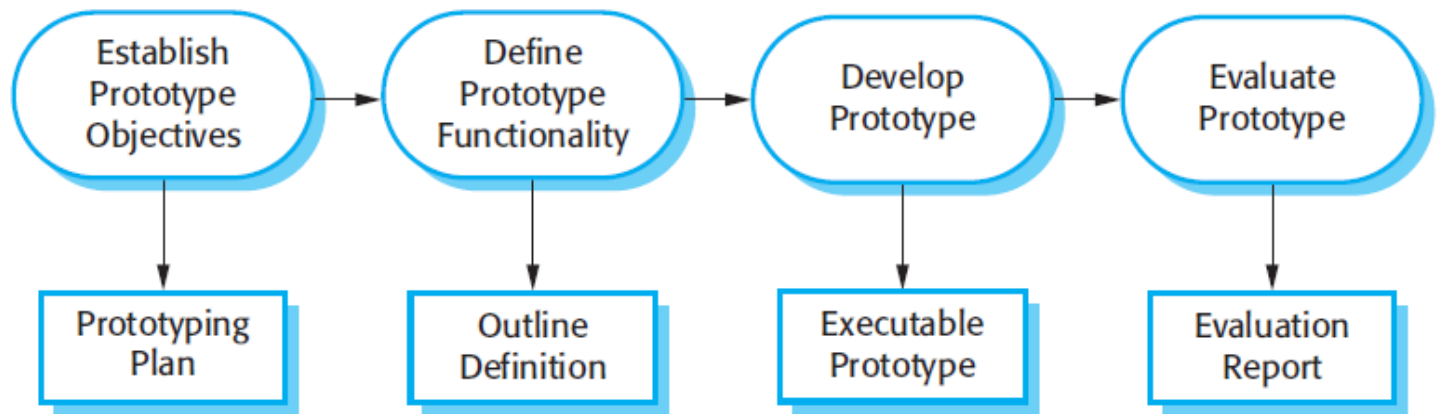
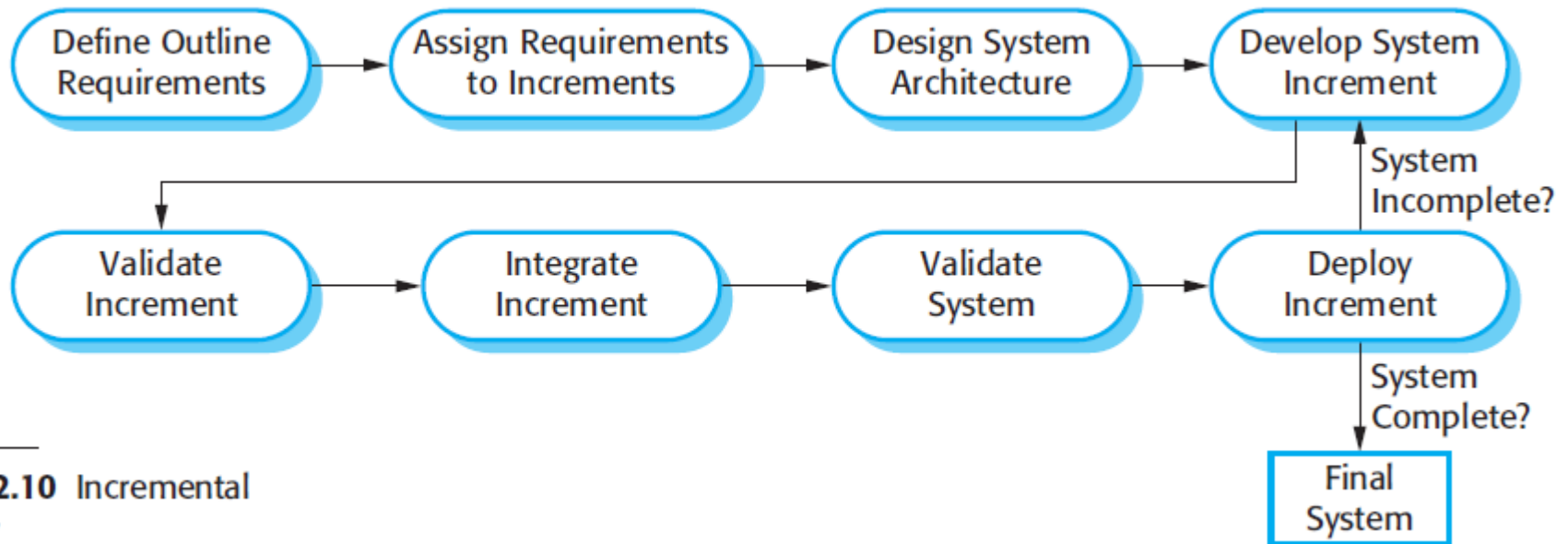


Figure 2.10 Incremental delivery



However, there are problems with incremental delivery:

1. Most systems require a set of basic facilities that are used by different parts of the system. As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments.
2. Iterative development can also be difficult when a replacement system is being developed. Users want all of the functionality of the old system and are often unwilling to experiment with an incomplete new system. Therefore, getting useful customer feedback is difficult.
3. The essence of iterative processes is that the specification is developed in conjunction with the software. However, this conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract. In the incremental approach, there is no complete system specification until the final increment is specified. This requires a new form of contract, which large customers such as government agencies may find difficult to accommodate.

The Rational Unified Process

The Rational Unified Process (RUP) (Krutchen, 2003) is an example of a modern process model that has been derived from **work on the UML** and the associated Unified Software Development Process (Rumbaugh, et al., 1999; Arlow and Neustadt, 2005).

The RUP recognizes that conventional process models present a single view of the process. In contrast, the RUP is normally described from three perspectives:

1. A dynamic perspective, which shows the phases of the model over time.
2. A static perspective, which shows the process activities that are enacted.
3. A practice perspective, which suggests good practices to be used during the process.

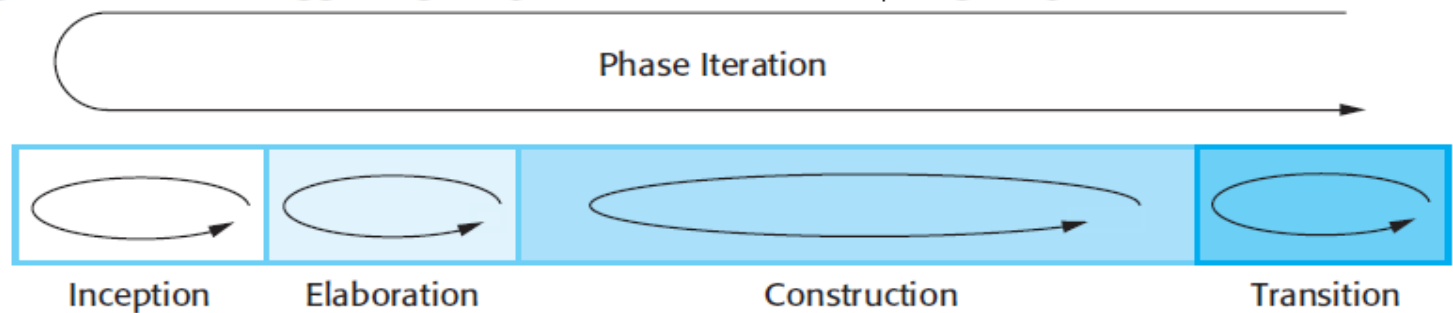


Figure 2.12 Phases in the Rational Unified Process

Inception The goal of the inception phase is to establish a business case for the system. You should identify all external entities (people and systems)

Elaboration The goals of the elaboration phase are to develop an understanding of the problem domain, establish an architectural framework for the system, develop the project plan, and identify key project risks.

Construction The construction phase involves system design, programming, and testing.

Transition The final phase of the RUP is concerned with moving the system from the development community to the user community and making it work in a real environment.

Workflow	Description
Business modelling	The business processes are modelled using business use cases.
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements.
Analysis and design	A design model is created and documented using architectural models, component models, object models, and sequence models.
Implementation	The components in the system are implemented and structured into implementation sub-systems. Automatic code generation from design models helps accelerate this process.
Testing	Testing is an iterative process that is carried out in conjunction with implementation. System testing follows the completion of the implementation.
Deployment	A product release is created, distributed to users, and installed in their workplace.
Configuration and change management	This supporting workflow manages changes to the system (see Chapter 25).
Project management	This supporting workflow manages the system development (see Chapters 22 and 23).
Environment	This workflow is concerned with making appropriate software tools available to the software development team.

Figure 2.13 Static workflows in the Rational Unified Process

Automated Process Support (CASE)

- 1 Computer-aided software engineering (CASE) is software used to support software development and evolution processes.
- 1 Activity automation
 - Graphical editors for system model development
 - Data dictionary to manage design entities
 - Graphical UI builder for user interface construction
 - Debuggers to support program fault finding
 - Automated translators to generate new versions of a program

Case Technology

- 1 Case technology has led to significant improvements in the software process, though not the order of magnitude improvements that were once predicted.
 - Software engineering requires creative thought - this is not readily automatable.
 - Software engineering is a team activity and, for large projects, much time is spent in team interactions. CASE technology does not really support these.

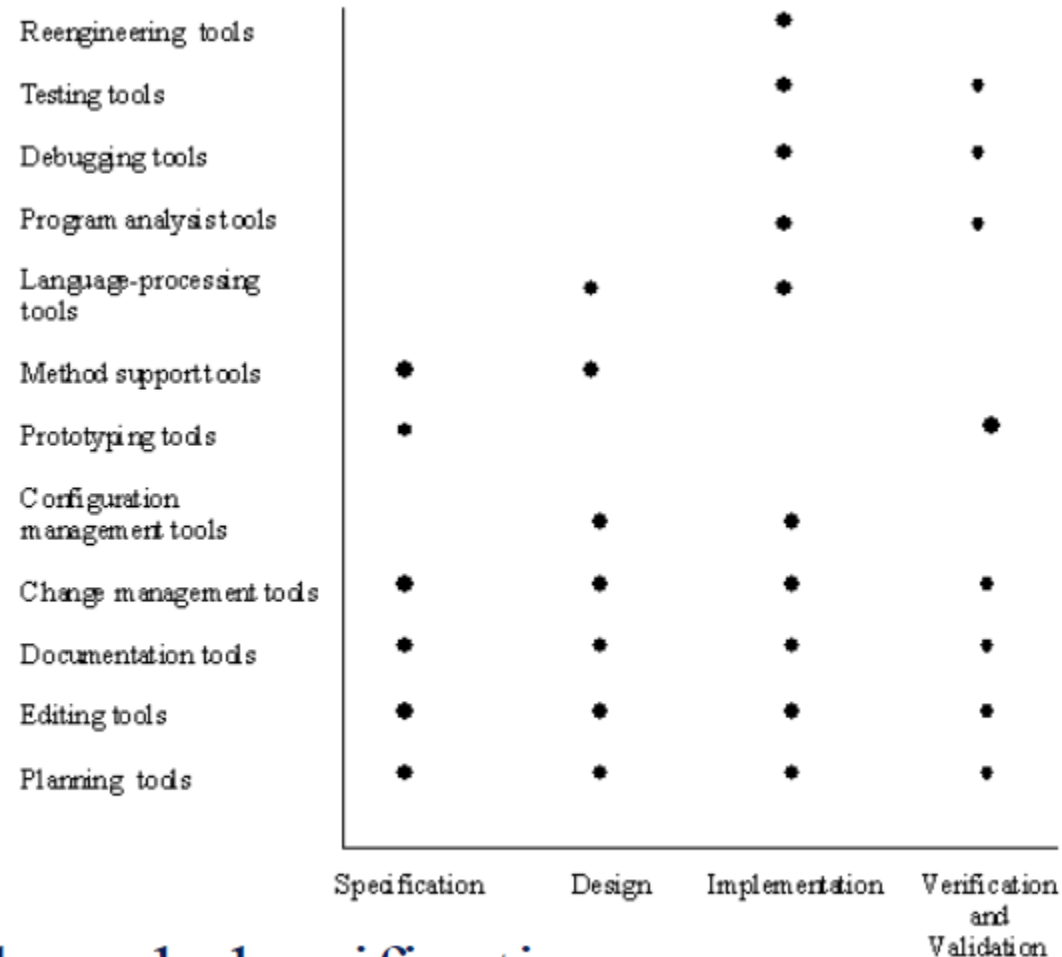
CASE Classification

Classification helps us understand the different types of CASE tools and their support for process activities.

- 1 Functional perspective
 - Tools are classified according to their specific function
- 1 Process perspective
 - Tools are classified according to process activities that are supported
- 1 Integration perspective
 - Tools are classified according to their organization into integrated units

Functional Tool Classification

Tool type	Examples
Planning tools	PERT tools, estimation tools, spreadsheets
Editing tools	Text editors, diagram editors, word processors
Change management tools	Requirements traceability tools, change control systems
Configuration management tools	Version management systems, system building tools
Prototyping tools	Very high-level languages, user interface generators
Method-support tools	Design editors, data dictionaries, code generators
Language-processing tools	Compilers, interpreters
Program analysis tools	Cross reference generators, static analysers, dynamic analysers
Testing tools	Test data generators, file comparators
Debugging tools	Interactive debugging systems
Documentation tools	Page layout programs, image editors
Re-engineering tools	Cross-reference systems, program re-structuring systems



Activity-based classification

Thank you